

I - Annexe – énoncés des TP

Les TP ci-après ne sont que des propositions (à caractère indicatif et non impératif).
Il ne faut pas hésiter à tester tout un tas de variantes (selon ses préférences personnelles).

NB: chaque nouvelle classe développée dans un TP devra être placée
dans un **package** adéquat (dont le nom est à choisir) .

1. TP (prise en main du jdk)

Objectif : Edition, Compilation et Exécution sans IDE , avec un simple éditeur de texte et le jdk :

A faire : Hello World !!!

2. TP (première classe simple, conventions JavaBean)

Objectif : Ecrire une classe Java dans les règles de l'art

A faire:

- Créer un nouveau projet java (basé sur la technologie "**maven**") de nom "**tpInit**" sous eclipse ou bien intelliJ (choisir une version LTSde java , ex : 21 ou 25)
- Créer un package "**tp**" (dans src/main/java).
- Créer la classe "**MyApp1**" avec une méthode **main()** qui servira aux tests.
- Créer (et tester) une première version d'une classe "**Personne**" avec des attributs "**nom**", "**age**", "**poids**" déclarés provisoirement "public".
- Coder une méthode "public void **afficher()**" qui affiche à l'écran/console les valeurs des attributs.
(+ premiers tests)
- Rendre "**private**" les **attributs** de la classe "Personne"
- Générer les méthodes **getXxx()/setXxx(...)** (+tests)
- Coder quelques **constructeurs**. (+tests)
- Coder ou générer la méthode classique "public String **toString()**" (provenant de la classe Object) en y construisant une chaîne de caractères complète regroupant tous les attributs + return
- Reprogrammer la méthode **afficher()** de façon à ce quelle appelle **toString()** en interne.
- Créer deux instances p1 et p2 de la classe **Personne** avec les mêmes valeurs internes.
- Comparer ces 2 instances et afficher si (oui ou non) les valeurs internes sont identiques.
- Reprogrammer ou générer la méthode "public boolean **equals(Object obj)**" sur la classe **Personne**. Cette méthode doit renvoyer "true" si et seulement si toutes les valeurs internes de this et de obj sont identiques.

Suite facultative du TP :

Coder également une classe "**Bagage**" (avec label, poids, volume) . Avec des poids en grammes (Integer) et des volumes en litres (Double) avec constructeurs, toString() , getter/setter , ...
Tester le bon comportement de la classe Bagage via une nouvelle méthode **testerBagage()** appelée par la méthode **main()** d'une nouvelle classe **MyApp2**.

Encore une fois la même chose !!! J'en ai marre de programmer des "get/set" pas passionnant disait le développeur "Jean Aimare" !!!

Heureusement, lombok est là pour vous simplifier la vie .

⇒ ajouter une dépendance lombok dans le pom.xml du projet

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.30</version> <!-- ou bien version plus récente -->
</dependency>
```

⇒ utiliser @Getter() @Setter() @ToString() @NoArgsConstructor() au dessus de la classe Bagage et commenter tous les équivalents générés par l'IDE eclipse ou intelliJ .

⇒ lancer un build maven via "mvn clean package" pour vérifier un bon fonctionnement maven.

⇒ les getters/setters générés par maven lors de la compilation sont normalement bien reconnus par l'IDE intelliJ qui propose d'installer automatiquement le plugin pour lombok.

⇒ avec l'IDE eclipse il faut effectuer une installation manuelle du plugin via le menu **Help / install new software** , sélectionner le chemin "**https://projectlombok.org/p2** " puis se laisser guider puis redémarrer eclipse , puis menu "**project clean**" .

3. TP (Tableaux, String, ...)

Code à placer dans **tp.MyApp2**

(Tous les tests liés aux personnes seront rangés dans tp.MyApp1 et **tout le reste** dans tp.MyApp2)

testTableauMoyenne() :

créer un tableau de 6 nombres réels
et calculer la moyenne

Suite facultative du Tp :

Trouver et afficher la plus grande des valeurs du tableau:

testString() :

String s1 = "2023-01-17";

//extraire la partie mois de différentes façons et afficher cette valeur

String chaine="YTREZA" ;

//créer une nouvelle chaine inverse où tous les caractères sont dans l'ordre inverse

//afficher la chaine inverse

4. TP static – constante , ...

- Déclarer une constante `Personne.AGE_MAJORITE` avec une valeur de 18
- Ajouter les éléments suivants sur la classe "Personne" :
 - * variable de classe privée "esperanceVie"
 - * méthodes de classe `getEsperanceVie()` et `setEsperanceVie()`
 - * méthode ordinaire `estMajeur()` retournant un booléen
- Tester la nouvelle version de `Personne` dans `tp.MyApp1`
- Déclencher le calcul racine carré de 81 dans une méthode de la classe `tp.MyApp2`

5. TP (Collections & Generics)

Essais très progressifs sur des collections élémentaires (de "Integer" ou de "String")

`tp.MyApp2.testListeString()` :

- créer une liste (ArrayList) de String
- ajouter quelques valeurs (ex : "janvier" , "février" , ...)
- parcourir une première fois la liste pour afficher les valeurs
- parcourir une seconde fois la liste pour transformer les éléments (ex : en majuscules)
- parcourir une seconde fois la liste pour afficher les valeurs modifiées
- afficher la taille de la liste
- tester d'éventuels ajouts et suppressions

Suite facultative du TP :

- créer une autre liste de Double (avec quelques valeurs) et calculer la moyenne
- faire tourner l'exemple du support de cours (`MyStack<T>`)

TP facultatif (mais conseillé) : enum Genre sur Personne :

Ajouter une énumération **Genre** (avec HOMME , FEMME , INDETERMINE comme valeurs possibles) au niveau de la classe `Personne` (partie additionnelle `private Genre genre` à gérer/manipuler/afficher) .

La valeur par défaut sera `INDETERMINE` et la précision du genre pourra être effectuée par une nouvelle variante du constructeur sans remettre en question les constructeurs existants.

TP facultatif (mais conseillé) sur saisies via Scanner :

Dans une sous fonction de type `tp.MyApp2.testScanner()` :

//on va demander à l'utilisateur de saisir des nombres x et y

//on va les additionner ensemble et afficher le résultat

//Autre idée de TP classique : générer un nombre entre 0 et 100 de manière aléatoire

//puis faire deviner ce nombre avec à chaque essai une indication : "plus petit" ou "plus grand"

//et une fois le nombre à deviner trouvé afficher le nombre d'essais.

6. TP POO (Héritage, Polymorphisme, interface)

Programmation orientée objet bien structurée: Héritage, Polymorphisme, Interface sur mini étude de cas (ex : dessin) **Objectif principal de cette mini étude de cas:**

Programmer une petite application java qui générera au moins un fichier "dessin.svg" affichable via un navigateur internet .

6.1. création et paramétrage du projet

Créer (avec IntelliJ ou eclipse) un nouveau projet "maven" simple intitulé "**dessin**"

groupId : tp ou autre

ArtifactId : dessin

Version de java : 21 ou 25 ou ... selon disponibilités.

Exemple de "properties" dans **pom.xml** :

```
<properties>
  <java.version>21</java.version>
  <maven.compiler.source>${java.version}</maven.compiler.source>
  <maven.compiler.target>${java.version}</maven.compiler.target>
  <maven.compiler.release>${java.version}</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

6.2. classe principale tp.MyApp

- générer un nouveau package "**tp**" dans src/main/java
- générer une nouvelle classe "**MyApp**" contenant la méthode principale **main()** dans le package "tp" .
- Au sein de la méthode main() afficher à la console le message **"cette application va générer un fichier dessin.svg"**
- Lancer une première fois l'exécution de l'application java et vérifier le bon fonctionnement au sein de la console :



```
<terminated> MyApp (2) [Java Application] C:\Program Files\Java\jdk-11.0.4\bin\javaw.exe (23 juin 2020 à 17:54:50)
cette application va générer un fichier dessin.svg
```

6.3. exemple de fichier ".svg"

créer à coté de pom.xml un nouveau fichier vide **exemple.svg**

Ouvrir ce fichier et placer (via un copier/coller) le contenu suivant :

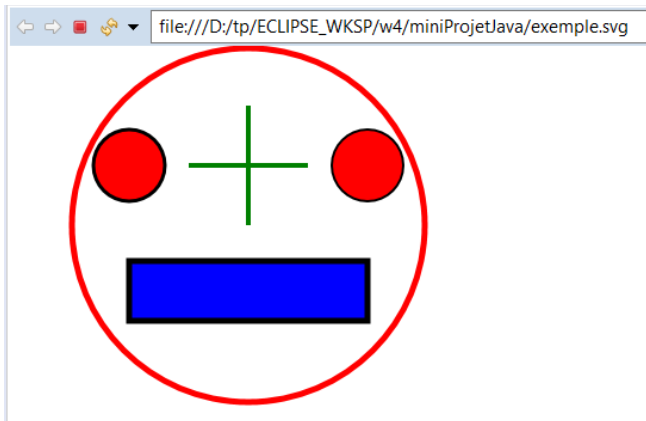
```
<svg xmlns='http://www.w3.org/2000/svg' height='400' width='500'>
  <circle cx='100' cy='100' r='30' stroke='black' stroke-width='3' fill='red' />
```

```

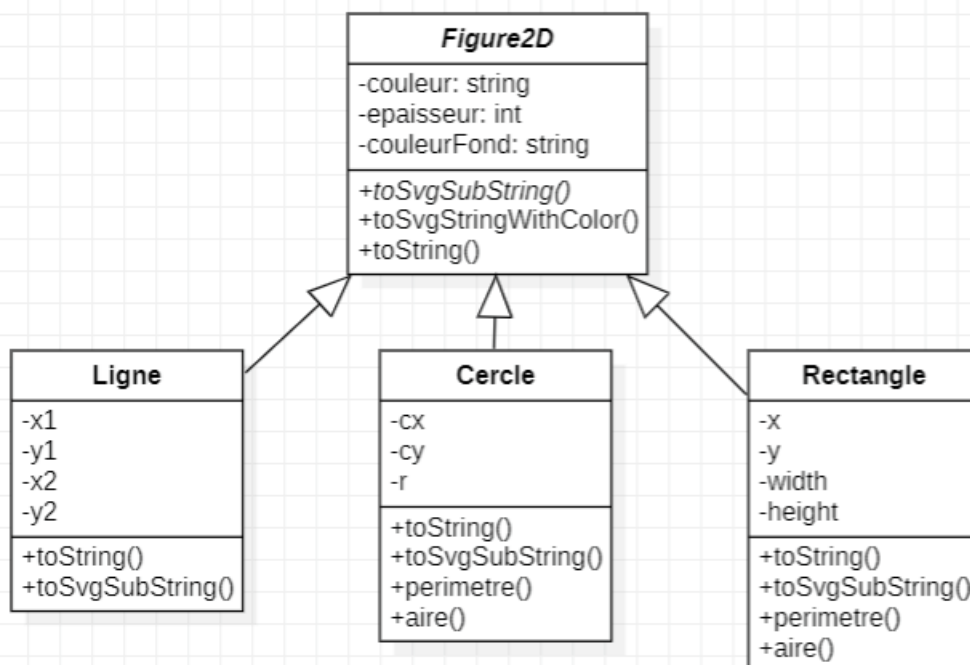
<line x1='200' y1='50' x2='200' y2='150' stroke='green' stroke-width='4' fill='none' />
<line x1='150' y1='100' x2='250' y2='100' stroke='green' stroke-width='4' fill='none' />
<circle cx='300' cy='100' r='30' stroke='black' stroke-width='2' fill='red' />
<rect x='100' y='180' width='200' height='50'
    fill='blue' stroke='black' stroke-width='5' />
<circle cx='200' cy='150' r='148' stroke='red' stroke-width='5' fill='none' />
</svg>

```

Visualiser ce dessin vectoriel au sein du navigateur web (ordinaire ou intégré à l'ide) :



6.4. première modélisation UML



6.5. première partie abstraite de tp.figure

créer (sous tp) le sous **package** **tp.figure**

créer (dans tp.figure) une *nouvelle classe abstraite* **"Figure2D"** avec le code initial suivant :

```
package tp.figure;

import java.io.ByteArrayOutputStream;
import java.io.PrintStream;

public abstract class Figure2D {
    protected String couleur="black"; //couleur du trait ou du contour ("black" par défaut)
    protected Integer epaisseur=1; //épaisseur du trait ou du contour (1 par défaut)
    protected String couleurFond; //null par défaut ("none" en svg)

    //design pattern "template method" avec polymorphisme sur sous tâche abstraite .
    public String toSvgStringWithColor() {
        //polymorphisme sur l'appel à this.toSvgSubString();
        //où this pourra référencer une instance de Cercle ou Ligne ou Rectangle
        String beginOfSvgString = this.toSvgSubString();

        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        PrintStream ps = new PrintStream(baos);
        ps.printf("%s stroke='%s' stroke-width='%d' fill='%s'/>",
                beginOfSvgString ,
                this.couleur ,
                this.epaisseur ,
                this.couleurFond==null?"none":this.couleurFond);

        return baos.toString();
    }
    public abstract String toSvgSubString();
    //+get/set , +constructeurs , +toString()
}
```

Compléter ce code en générant via les assistants de l'IDE les éléments habituels
(+get/set , +constructeurs , +toString())

6.6. partie concrète de tp.figure

Créer au sein de tp.figure , 3 nouvelles classes concrètes :

- Ligne , Cercle , Rectangle héritant de Figure2D

NB :

- Chacune de ces classes devra comporter au minimum 2 constructeurs :
 - un constructeur par défaut (avec zéro argument)
 - un constructeur avec toutes les coordonnées comme premiers arguments importants

et avec les couleur, epaisseur et couleurFond comme derniers arguments secondaires

Coordonnées internes d'une ligne :

```
private int x1;
private int y1;
private int x2;
private int y2;
```

Coordonnées internes d'un cercle :

```
private int cx;//x du centre du cercle
private int cy;//y du centre du cercle
private int r; //rayon
```

Coordonnées internes d'un rectangle :

```
private int x;
private int y;
private int width;
private int height;
```

NB : On pourra coder les méthodes imposées petit à petit en les testant au fur et à mesure de leur mise en oeuvre.

Autrement dit , certaine méthodes pourront temporairement être laissées avec le code par défaut généré par les assistants de l'IDE .

Les méthodes délicates `toSvgSubString()` seront codées et testées ultérieurement .

6.7. premiers tests via la classe tp.MyApp

Coder au sein de tp.MyApp quelques tests de ce genre :

```
Rectangle r = new Rectangle(100,180,200,50,"black",5,"blue");
System.out.println(r.toString());
System.out.println("r.perimetre="+r.perimetre());
System.out.println("r.aire="+r.aire());
```

//idem pour Cercle

//idem pour Ligne (sans périmetre ni aire)

6.8. coder et tester les méthodes `toSvgSubString()`

La méthode `.toSvgSubString()` de la classe **Ligne** devra générer et retourner une chaîne de caractère de ce type : `<line x1='200' y1='50' x2='200' y2='150'`

La méthode `.toSvgSubString()` de la classe **Cercle** devra générer et retourner une chaîne de caractère de ce type : `<circle cx='300' cy='100' r='30'`

La méthode `.toSvgSubString()` de la classe **Rectangle** devra générer et retourner une chaîne de caractère de ce type : `<rect x='100' y='180' width='200' height='50'`

NB : étant donné que la méthode `toSvgStringWithColor()` héritée de la classe abstraite **Figure2D** ajoute une partie finale de de genre `stroke='black' stroke-width='3' fill='red' />`

on testera indirectement la méthode `.toSvgSubString()` en appelant `.toSvgStringWithColor()` sur des lignes , des cercles et des rectangles .

exemple :

```
System.out.println(r.toSvgStringWithColor());
```

6.9. Polymorphisme avec version préliminaire de MySvgUtil

- créer un nouveau package **tp.svg**
- créer et coder le début de la nouvelle classe **MySvgUtil** suivante :

```
package tp.svg;

import java.util.List;

import tp.figure.Figure2D;

public class MySvgUtil {

    public static String generateGlobalSvgContent(List<Figure2D> listeFig) {
        //utiliser un buffer sous forme d'une instance de la classe StringBuilder

        //ajouter au buffer la première ligne d'un bloc svg
        //<svg xmlns='http://www.w3.org/2000/svg' height='400' width='500'>

        //boucler sur chaque élément de la liste listeFig
        //et ajouter au buffer une ligne dont la valeur est construite
        //via la méthode .toSvgStringWithColor() appelant indirectement
        //la méthode polymorphe .toSvgSubString()

        //ajouter au buffer la dernière ligne d'un bloc svg
        //</svg>

        //retourner la transformation du buffer en String
    }
}
```


Pour tester cette classe , on ajoutera dans la méthode **main()** de **tp.MyApp** du code qui :

- créera une instance de `ArrayList<Figure2D>`
- y ajoutera quelques instances de `Cercle` , `Ligne` et `Rectangle`
- appellera la méthode statique . **generateGlobalSvgContent** de **MySvgUtil** .

On pourra enfin :

- recopier le texte généré à la console dans un fichier temporaire `mySvg.svg`
- visualiser ce fichier *mySvg.svg* au sein d'un navigateur internet

Une future suite de ce Tp permettra de compléter le code de la classe `MySvgUtil` de manière à directement générer un nouveau fichier SVG .

6.10. Seconde partie abstraite de tp.figure

Au sein du package `tp.figure` , coder les 2 interfaces suivantes :

```
public interface Surface {  
    public double perimetre();  
    public double aire();  
}
```

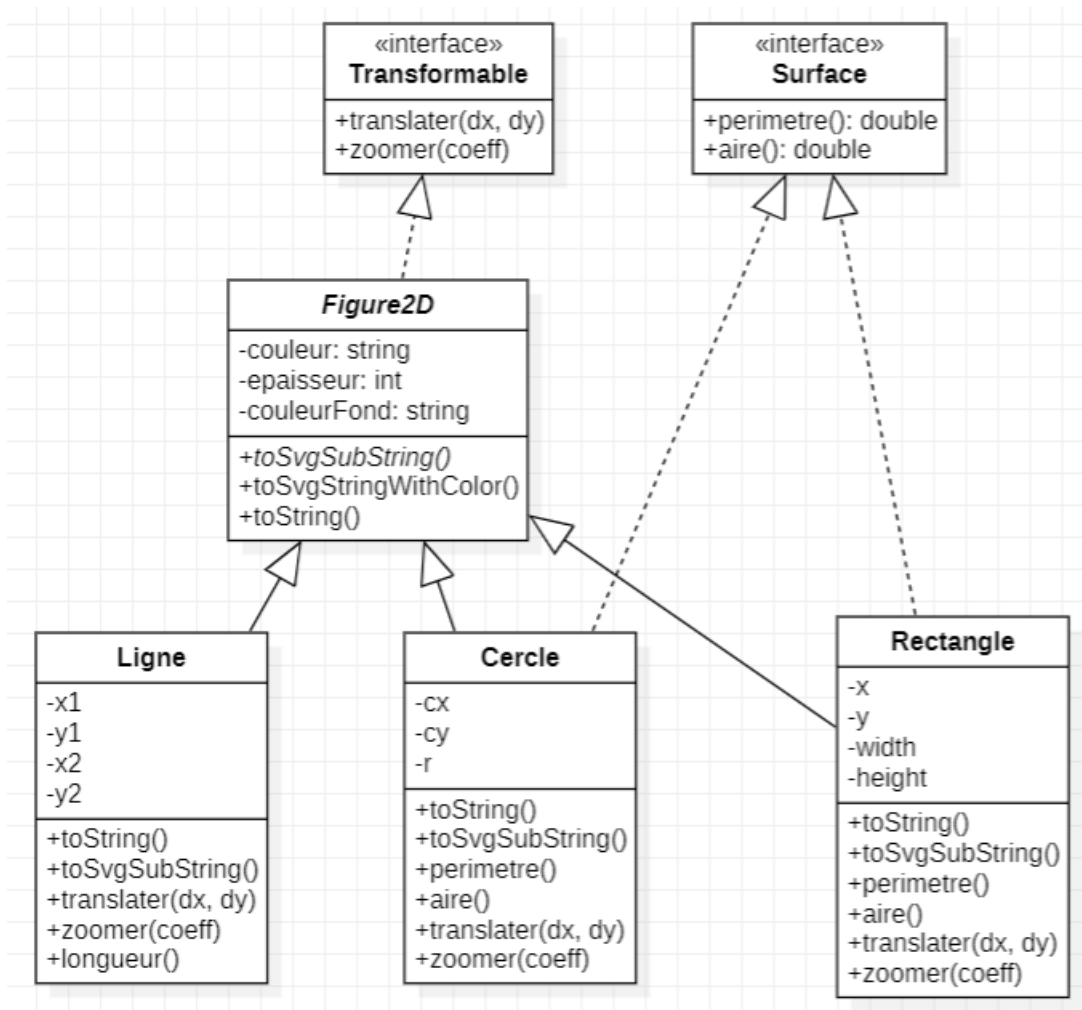
et

```
public interface Transformable {  
    public void translater(int dx,int dy);  
    public void zoomer(double coeff);  
}
```

Restructurer *la classe abstraite* "**Figure2D**" de telle manière qu'elle implémente l'interface **Transformable** .

Les 3 classes concrètes `Cercle`,`Ligne` et `Rectangle` devront également coder les méthodes imposées par l'interface `Transformable` (indirectement récupérée par l'héritage de `Figure2D` déclarant implémenter l'interface `Transformable`) .

Restructurer les classes **Cercle** et **Rectangle** de manière à ce qu'elles implémentent explicitement l'interface **Surface** .



Suggestion de polymorphisme exploitant la nouvelle structure :

Au sein d'une nouvelle sous méthode statique `MyApp.changeFigures(List<Figure2D> listeFig)` appelée par `MyApp.main()`, ajouter quelques lignes de code de manière à :

- calculer une première fois la somme de tous les périmètres des surfaces et des longueurs des lignes
- déclencher en boucle sur `listeFig` une translation (`dx=30, dy=20`) de toutes les figures
- recalculer une seconde fois la somme de tous les périmètres des surfaces et des longueurs des lignes (et vérifier que la valeur totale n'a pas été changée)
- déclencher en boucle sur `listeFig` un zoom (`coeff=2`) de toutes les figures
- recalculer une seconde fois la somme de tous les périmètres des surfaces et des longueurs des lignes (et vérifier que la valeur totale est maintenant deux fois plus grande)

7. TP - Premier traitement d'exception

Ré-ouvrir (avec l'IDE IntelliJ ou Eclipse) le projet initial "**tpInit**" (celui où avait été codé la classe `Personne`).

améliorer le code de la méthode `.setAge()` de la classe **Personne** en soulevant une exception de type **IllegalArgumentException** (héritant de `RuntimeException`) via le mot clef **throw** en cas d'âge négatif invalide et tester le tout via au moins un **try/catch** au sein de `MyApp1.testPersonne()`.

8. TP - Second traitement d'exception

Ré-ouvrir (avec l'IDE IntelliJ ou eclipse) le projet "dessin" (celui de la mini étude de cas) .

8.1. Restructurer le code de tp.MyApp

```
public class MyApp {

    public static void main(String[] args) {
        double coeffZoom = 1;
        int dx=0;
        int dy=0;
        String typeFig=null;
        //premiersTests();
        //.....du code sera à ajouter ici ...
        enchaînerTransformationsEtGenerationFichierSvg(coeffZoom,dx,dy,typeFig);
    }

    public static List<Figure2D> buildListeFigures() {
        List<Figure2D> listeFigures = new ArrayList<>();
        listeFigures.add(new Rectangle(100,180,200,50,"black",5,"blue"));
        listeFigures.add(new Ligne(150,100,250,100,"green",4/*,null*/));
        listeFigures.add(new Cercle(100,100,30,"black",3,"red"));
        return listeFigures;
    }

    public static void enchaînerTransformationsEtGenerationFichierSvg(double coeffZoom ,
                                                                    int dx, int dy , String typeFig) {
        List<Figure2D> listeInitialeFigures = buildListeFigures();
        List<Figure2D> listeTransformeeFigures =
            listeInitialeFigures.stream()
                //effacter une premiere transformation de type zoomer(coeffZoom)
                //effacter une seconde transformation de type translation(dx,dy)
                //filtrer selon le type de figure (typeFig , ex : "Cercle") via un test de type instanceof ...
                .collect(Collectors.toList());
                //futur MySvgUtil.generateSvgFile(listeTransformeeFigures, "dessin2.svg");
                MySvgUtil.generateGlobalSvgContent(listeTransformeeFigures);
    }

    public static void premiersTests() {
        //déplacer ici les tests effectués au sein de la partie 1
    }
}
```

Lancer une première fois le code restructuré pour vérifier qu'il y a toujours une génération de lignes au format svg sur la console texte.

8.2. Récupération des éventuels arguments de la ligne de commande avec traitement d'exception

Ajouter du code dans la fonction principale **MyApp.main()** de manière à récupérer les valeurs des arguments facultatifs de la ligne de commande :

```
java tp.MyApp --coeffZoom=1.5 --dx=10 --dy=20 --typefig=Cercle
```

On se contentera dans un premier temps d'afficher les valeurs choisies (ou par défaut)

Dans un second temps , on affichera un message d'erreur de type "la valeur de dx est invalide (pas numérique) si Integer.parseInt(...) soulève une exception (idem pour dy et pour coeffZoom avec Double.parseDouble(...))

9. TP (logs , test unitaires) à éventuellement différer

A éventuellement mettre en place (en différé) une fois de JUnit5 et slf4j auront été abordés en cours :

- Réorganisation de certains packages (refactor)
- Génération d'un .jar et démarrage d'une application java en ligne de commande
- petit test unitaire élémentaire avec JUnit5 (@Test , @BeforeEach , assertTrue())
- génération de ligne de logs via slf4j et log4j2
- sensibilisation aux mocks (une démonstration "Mockito" sera idéalement effectuée par le formateur)

A faire dans au moins une des 2 applications des Tps ("tpInit" et/ou "dessin")

10. TP tris et lambda, stream

10.1. Tri et transformation de personnes

Ré-ouvrir (avec l'IDE IntelliJ ou eclipse) le projet initial "tpInit" (celui où avait été codé la classe Personne) .

Dans une sous fonction de type **MyApp1.testCollectionPersonne()** :

- créer une liste de Personne , y ajouter quelques valeurs
- trier cette liste par noms croissants puis afficher cette liste modifiée
- trier une nouvelle fois cette liste par ages décroissants puis afficher cette liste modifiée
- **Dans un second temps , une fois la partie 1 testée, refaire ça avec des lambda expressions**

Refaire cela avec des streams() et en enchaînant "filtrage des personnes majeures" , "tri selon age"

puis transformations avec noms en majuscules dans une méthode *MyApp1.testStream()* :

10.2. Tri et transformation de figure2D

Ré-ouvrir (avec l'IDE IntelliJ ou Eclipse) **le projet "dessin"** (celui de la mini étude de cas) .

Coder les transformations et filtrages suggérés par les commentaires de
`enchaînerTransformationsEtGenerationFichierSvg()`

Tester cela via un affichage graphique au sein d'un navigateur d'un fichier temporaire *mySvg2.svg*
comportant une copie des lignes générées à la console

11. TP facultatif (LocalDate & LocalTime)

Ré-ouvrir (avec l'IDE IntelliJ ou eclipse) le projet initial "tpInit"

Ajouter dans tp.MyApp2 une sous méthode du main() , par exemple "testDates()" qui :

- demandera à saisir deux dates
- affichera l'écart en nombre de jours et/ou de mois

12. TP (Gestion des fichiers) :

12.1. Génération de fichier SVG sur application dessin

Ré-ouvrir (avec l'IDE IntelliJ ou eclipse) le projet "dessin" (celui de la mini étude de cas) .

- Ajouter la méthode `generateSvgFile(...)` au sein de la classe `svg.MySvgUtil` :

```
package tp.svg;

import java.util.List;

import tp.figure.Figure2D;

public class MySvgUtil {

    public static String generateGlobalSvgContent(List<Figure2D> listeFig) {
        //déjà codé et testé
    }

    public static void generateSvgFile(List<Figure2D> listeFig , String fileName) {
        //ex de fileName : "dessin.svg"

        //ouvrir le fichier en écriture (flux élémentaire + PrintStream)

        //écrire dans ce fichier le résultat de la sous méthode generateGlobalSvgContent()

        //fermer les flux ouverts
    }

}
```

Pour tester cette classe , on appellera la méthode `MySvgUtil.generateSvgFile` (plutôt que `MySvgUtil.generateGlobalSvgContent`) dans méthode adéquate de `tp.MyApp` (`main()` ou `enchaînerTransformationsEtGenerationFichierSvg()` ou `premiersTests()`)

On pourra enfin :

- vérifier l'existence d'un fichier directement généré (*dessin.svg* ou autre) via un **Refresh** IntelliJ ou eclipse

- visualiser le fichier généré via un navigateur internet

12.2. long TP facultatif: lecture et écriture de fichier CSV

Ré-ouvrir (avec l'IDE intelliJ ou eclipse) le projet initial "tpInit"

Coder (par ajout) :

- Classe Produit(numero, label, categorie, prix, poids)
- Classe Stat(nbProduits, moyennePrix, moyennePoids)
- Classes MyCsvApp et MyCsvUtil permettant de :
 - lire un fichier produits.csv et générer un fichiers stats_products.csv

produits.csv

```
num;label;categorie;prix;poids
1;cahier;papeterie;3.6;0.123
2;pommes;nourriture;2.6;1.0
3;poires;nourriture;3.69;1.2
4;stylo;papeterie;2.6;0.023
5;bananes;nourriture;1.79;1.3
6;gomme;papeterie;1.6;0.019
```

stats_products.csv

```
nbProduits;moyennePrix;moyennePoids
6;2.646666666666667;0.6108333333333333
```

Pleins d'extensions/suites facultatives pour ce TP :

- générer des fichiers triés
- générer un fichier .csv en utilisant de l'introspection / réflexion au sein de la classe MyCsvUtil
- coder une nouvelle annotation @CsvIgnore et l'utiliser au sein de MyCsvUtil pour ignorer certaines colonnes au sein d'un fichier .csv à générer

Encore une autre suite facultative à ce TP:

- ajouter la dépendance maven "jackson-databind" et générer un fichier .json à partir d'un objet java (de type "Produit") ou vice versa .

13. TP (Accès aux bases de données via JDBC):

Variantes proposées pour ce TP (en choisir une):

<i>Variante</i>	<i>1</i>	<i>2</i>
Application	tpInit	dessin
Nom de base de données	person_db	drawing_db
Nom table principale	personne (id,nom,age,poids)	dessin(id,titre,path,description)
Classe java (de données)	PersonEntity héritant de Personne (nom,age,poids)	Dessin(id,titre,path,description) où path sera soit un nom de

	et ajoutant un champ id de type Long qui sera considéré comme la clef primaire	fichier svg , soit une url d'un fichier téléchargeable en ligne
suggestion	Test JUnit	Test JUnit + éventuelle insertion d'un nouvel enregistrement lors d'une nouvelle génération de fichier svg

Utiliser l'api JDBC pour se connecter à une base de données relationnelles comportant une table de personnes ou de dessins . On codera au minimum une lecture et éventuellement les opérations standards "CRUD".

Indications :

- installer si besoin "**mariaDB**" (version "open source" de MySQL facile à installer sous windows ou linux) , préparer éventuellement une base de données via un script SQL **init_db.sql** (avec CREATE TABLE et quelques INSERT INTO ...)
- ajouter une dépendance maven permettant de se connecter à MySql (ou bien à MariaDB compatible).
- Structurer proprement le code d'accès aux données avec interface "PersonneOuDessinDao" et classe d'implémentation "PersonneOuDessinDaoJdbc"
- Utiliser un fichier **db.properties** (à idéalement placer dans src/main/resources) pour ne pas coder en dur les paramètres de connexions : (username, password, url, ...).
- Utiliser tout cela dans une classe de test (exemple : **TestJdbcDao**) en commençant par coder et lancer un test de lecture .

Exemple de dépendance maven :

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>26.7.0</version>
</dependency>
```

ou bien

```
<dependency>
  <groupId>org.mariadb.jdbc</groupId>
  <artifactId>mariadb-java-client</artifactId>
  <version>3.5.10</version>
</dependency>
```

14. TP (Gestion élémentaire des threads) :

Ré-ouvrir (avec l'IDE intelliJ ou eclipse) le projet initial "**tpInit**"

Tester le démarrage d'un nouveau Thread qui pourra :

- afficher un premier message "café en préparation"
- faire une pause de 5s
- afficher un second message "café prêt"

15. TP (syntaxes java récentes):

Sur application "**tpInit**", expérimenter (au sein d'une classe de test JUnit) différentes manières de générer un fichier **produit.json** :

- depuis une classe java **Produit(numero, label, categorie, prix, poids)** ou depuis un **record** équivalent .
- avec l'api jackson-databind ou bien avec des **text-blocs**

On pourra expérimenter une augmentation de prix paramétrée par une **switch expression** basée sur l'analyse de la catégorie .

On pourra éventuellement restructurer certains morceaux de codes des TP réalisés précédemment en ajoutant de type de code (ou d'améliorations) :

- mot clef "var"
- pattern matching , ...
- virtual thread

16. TP (debug et monitoring de JVM):

- Effectuer avec l'IDE un debug pas à pas (avec un point d'arrêt)
- Utiliser JConsole pour visualiser quelques éléments de la JVM (nb threads , mémoire consommée, ...) lorsque l'application sera en cours de fonctionnement (et pas encore arrêtée , par exemple en attente de saisie) .